

BugLens SDK

Android Bug Reporting & Crash Monitoring Platform

"Reduce bug resolution time by automatically collecting the context developers need to reproduce and fix issues."

PROJECT MODEL

SDK + BACKEND + DEVELOPER PORTAL

Problem & Solution

The Problem

Mobile developers often struggle with incomplete, non-actionable bug reports from users and QA teams.

- Vague text descriptions lacking reproduction steps
- No visual attachments (screenshots or recordings)
- Missing critical hardware, OS, and memory states
- Uncaught crashes lack complete, localized context

The BugLens Solution

An integrated SDK that captures contextual diagnostics the moment an issue occurs in-app.

1. • **User encounters bug / shake trigger**
2. • **SDK extracts device & session state**
3. • **Direct sync to Firestore and Storage**
4. • **Developer views triage diagnostics**

System Architecture

Backend Responsibilities

- Store Bug Reports
- Store Screenshots
- Query Reports
- User Metadata
- Store Crash Reports
- Authenticate Devs
- Manage Status Updates
- Real-Time Sync

Note: Firebase acts as a Backend-as-a-Service (BaaS) platform and fulfills all persistence, authentication, synchronization, storage, and retrieval requirements.

Architectural Highlights

- **Serverless Pipeline:** Zero custom backend maintenance needed by utilizing secure, direct-to-Firebase endpoints.
- **Security & Auth:** Built-in Firebase Authentication handles developer session access.
- **Optimized Payloads:** Heavy image files are uploaded directly to Object Storage, saving document write bandwidth.

SDK Capabilities

Telemetry & Context

- Device & OS details
- Network & battery status
- Dynamic screenshots
- User session tags

Automation Hooks

- Global JVM Crash Handler
- Hardware Shake listener
- Lifecycle state tracking
- Network monitor

Storage & Sync

- Local Room database cache
- Auto-retries on network loss
- Asynchronous processing
- Connectivity Resilience

Resiliency: All reports are resilient to connectivity failures through local persistence and background synchronization.

Report a Bug

Title

Description

Screenshot Preview

Submit Report

Public SDK API

```
// Initialize inside application context
BugLens.init(context, "API_KEY_#1337") // Activate
optional reporting interceptors
BugLens.enableCrashReporting()
BugLens.enableShakeToReport() // Attach runtime
contextual parameters BugLens.setUserId("eyal_fabian")
BugLens.setMetadata("tier", "premium")
```

Function	Behavior
init()	Loads parameters, starts SQLite cache
showReporter()	Launches manual UI model window
setMetadata()	Attach custom contextual info
setUserId()	Tethers reports to concrete accounts

"The SDK was intentionally designed to require minimal integration effort and follows a developer-first API philosophy."

Internal SDK Components

Data Managers

Report Manager: Constructs structured schemas from input views and screenshots.

Screenshot Manager: Captures host app frame layout on trigger.

Core Engines

Queue Manager: Orchestrates disk persistence to prevent memory leaks.

Crash Handler: Catches uncaught exceptions on default runtime threads.

Network Layer

Upload Manager: Interacts securely with Remote Firestore and Cloud Storage.

Firebase Client: Wrapper for authenticated SDK and database actions.

INTERNAL REPORTING WORKFLOW



Each module follows the **Single Responsibility Principle**, allowing independent maintenance and future scalability.

Android System Feasibility

No-Permission API Tracking

The SDK extracts diagnostics using standard APIs that require zero user-permission prompt overlays:

- **android.os.Build:** Product model, manufacturer, and API version.
- **ActivityManager.MemoryInfo:** Available RAM limits and memory-state indicators.
- **ConnectivityManager:** Simple active network interface transport checks.

Additional Data Points

- App Version
- Screen Resolution
- Network Type
- Device Language
- Battery %
- Timestamp

Crash Exception Strategy

```
// Intercept thread safety val originalHandler =  
Thread.getDefaultUncaughtExceptionHandler()  
Thread.setDefaultUncaughtExceptionHandler { thread, exception →  
    saveCrashToDisk(exception)  
    originalHandler?.uncaughtException(thread, exception) }
```

All collected information is accessible through public Android SDK APIs and does not require sensitive runtime permissions.

Database Design & Optimization

Report Document (JSON)

```
{ "reportId": "rep_987abc", "appId": "com.eyal.myapp", "userId":  
"user_21", "status": "OPEN", "deviceModel": "Pixel 7 Pro", "manufacturer":  
"Google", "androidVersion": "13", "appVersion": "1.0.4", "networkType":  
"WIFI", "batteryLevel": 72, "screenshotUrl": "gs://bucket/scr_987.jpg",  
"createdAt": "2024-05-01T17:19:15" }
```

OBJECT RELATIONSHIP:

Application → Reports [Metadata, Screenshot URL] → Crash Reports

Why Firestore?

- Flexible document schema for dynamic metadata
- No complex joins required for nested logs
- Real-time dashboard updates via Snapshots

Query Optimization

Composite indexes enable high-speed filtering and pagination:

appId + createdAt

appId + status

appId + appVersion

These indexes allow the developer portal to scan millions of records in milliseconds.

Developer Dashboard

Portal Functions

- View Reports
- Filter by Device
- Filter by Status
- View Crash Logs
- Secure Login
- Search Reports
- Filter by Version
- View Screenshots
- Change Status

Technology Stack

React

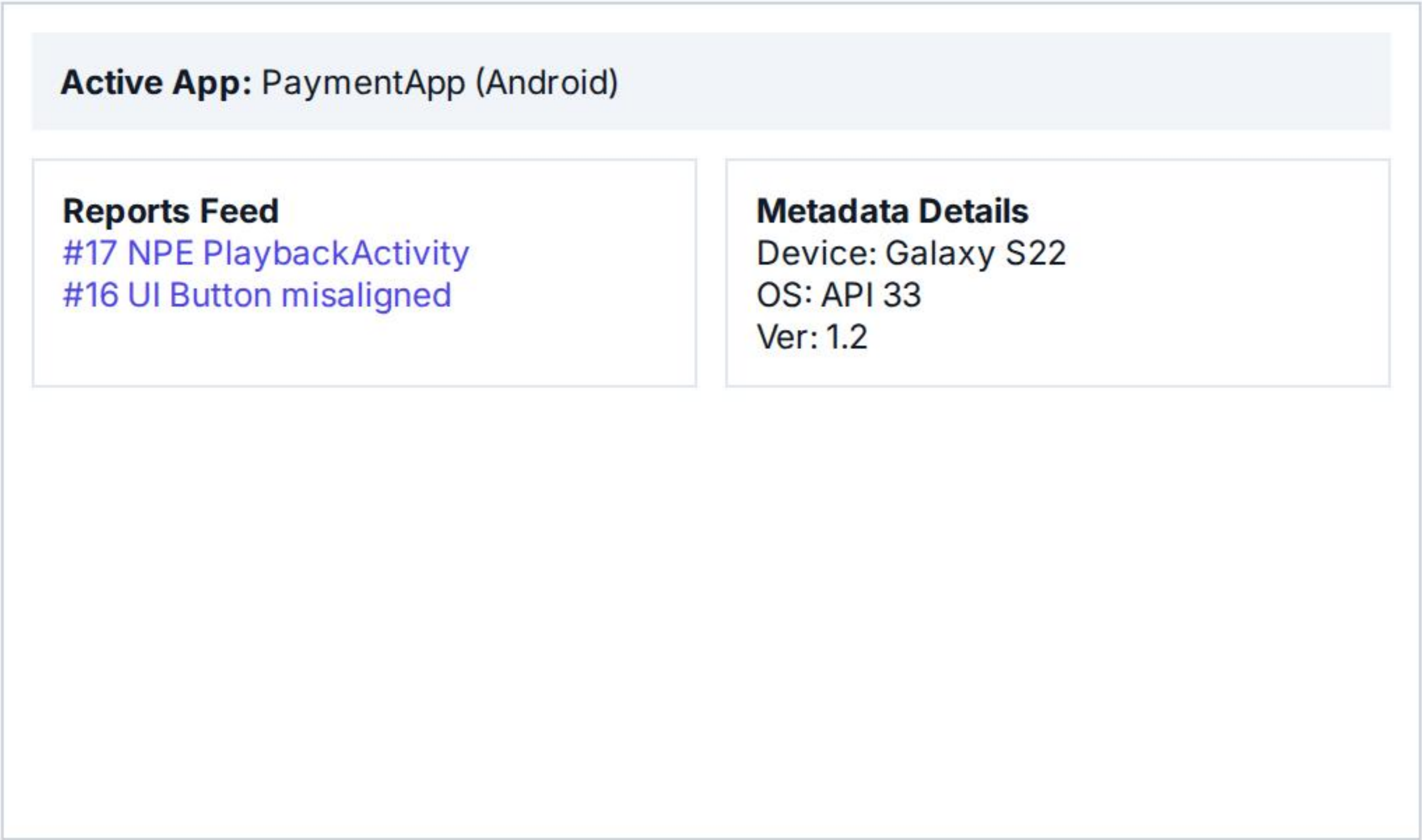
Vite

Firebase Auth

Firestore

Firebase Hosting

PORTAL MOCKUP DIAGRAM



"The portal serves as the primary debugging environment where developers investigate, classify, and resolve incoming reports."

Workflow & Future Roadmap

Offline Synchronization

1. **Create:** User triggers report
2. **Persist:** Store in Room DB
3. **Worker:** WorkManager backgrounds job
4. **Upload:** Push to Firebase
5. **Clean:** Delete local copy

Technical Design Decisions

- **Firebase:** Eliminate infrastructure costs.
- **Room DB:** Ensure reliability during offline ops.
- **WorkManager:** Guarantee background sync.

Planned Enhancements

- **AI Classification:** Automatic categorization of bugs.
- **Duplicate Detection:** Merge similar reports automatically.
- **ANR Detection:** Monitor Main Thread hang events.
- **Smart Prioritization:** Rank issues by user impact.

Ready for Development

A complete ecosystem for high-fidelity Android bug reporting.